

Reviewer Pack — Instance-Complexity Certification Depth Bounds Monotone DNF ...

Entry 17e6b7435e5a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 11:12:10 UTC

Instance-Complexity Certification Depth Bounds Monotone DNF Size for k-CLIQUE

Entry ID: 17e6b7435e5a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 11:12:10 UTC

1. Conjecture

Field A (mathematical branch): Algorithmic instance complexity (Orponen-Ko-Schöning-Watanabe), realized as resource-bounded conditional Kt approximated by greedy DPLL-style minimal-certificate length on a fixed monotone DNF **Field B** (complexity object): Monotone DNF formula size for the k-CLIQUE indicator (Razborov approximation-method setting, edges as variables)

Statement:

Let F be a monotone DNF over $N = C(n, 2)$ edge variables on graphs with n vertices, and let D_k be the uniform distribution on assignments of Hamming weight exactly $C(k, 2)$. For x in $\{0, 1\}^N$ define $\text{cert}(F, x) :=$ the number of variables revealed by a fixed deterministic greedy certifier (which iteratively reveals the variable maximizing the number of live monotone terms it touches, terminating once $F(x)$ is forced) before $F(x)$ is determined; set $\mu(F) := E_{x \sim D_k}[\text{cert}(F, x)]$. Conjecture: (i) for every monotone DNF F of size $\leq N^c$ with $c \leq 3$, $\mu(F) \leq c \cdot \log_2(N) + 8$; and (ii) for every monotone DNF F that computes the k-CLIQUE indicator at $k = \lceil \sqrt{n} \rceil$, $\mu(F) \geq \lfloor n/4 \rfloor$. A single (F, k, n) violating either bound on at least one of 30 seeds refutes the conjecture.

Rationale (proposer's reasoning):

Instance complexity is a meta-complexity invariant that, unlike single-number truth-table properties, is defined on (formula, input) pairs and inherits non-largeness from K-complexity, so it sidesteps Razborov-Rudich while still being a hard combinatorial witness; the greedy certificate is its concrete computable proxy. Poly-size monotone DNFs admit cheap certificates (one quickly-falsifying 0 or one fully-set term), giving the $O(\log N)$ ceiling; whereas at the clique threshold $k \approx \sqrt{n}$, any monotone DNF computing k-CLIQUE must, in expectation, expose $\Omega(n)$ edges before disambiguating clique from non-clique because both planted-clique and locally-cliquey near-misses survive every short partial assignment. Bridging instance complexity to Razborov's monotone setting is rarely attempted (< 5 papers) and would furnish a submodular DNF measure directly meeting the focus's (i, ii, iii).

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e48b30e437e916d2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{6, 7, 8, 9, 10\}$, $k \in \{3, 4\}$, and $F \in \{F_{\text{clique}}, F_{\text{rand}} \text{ with } m \in \{10, 30, N^2\}\}$, run 30 seeds each. Support requires: (i) $\mu(F_{\text{rand}}) \leq 3 \cdot \log_2(N) + 8$ on $\geq 95\%$ of seeds per (n, k, m) ; (ii) $\mu(F_{\text{clique}}) \geq \lfloor n/4 \rfloor$ on $\geq 95\%$ of seeds per (n, k) ; (iii) mean $\mu(F_{\text{clique}})/\mu(F_{\text{rand}})$ strictly increases in n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - monotone DNF size k-clique lower bound approximation method Razborov - decision tree certificate complexity monotone DNF average-case random k-clique - instance complexity Kt resource-bounded certificate DPLL monotone formula

Top relevant hits considered: - [<http://arxiv.org/abs/1102.2932v2>] Applications of Monotone Rank to Complexity Theory - [<http://arxiv.org/abs/2211.03396v3>] Certificate Games and Consequences for the Classical Adversary Bound - [<http://arxiv.org/abs/cs/0209032v3>] Complexity Results on DPLL and Resolution

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A, b):
        n = len(b)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
```

```

        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            A[j][i:] = [A[j][k] - factor * A[i][k] for k in range(i, n)]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def matrix_multiply(A, B):
    m, k = len(A), len(B[0])
    result = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            result[i][j] = sum(A[i][l] * B[l][j] for l in range(len(B)))
    return result

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def inverse(A):
    n = len(A)
    det_A = determinant(A)
    if det_A == 0:
        raise ValueError("Matrix is singular")
    adjoint = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]
            cofactor = determinant(submatrix) * (-1) ** (i+j)
            adjoint[j][i] = cofactor
    return matrix_multiply(adjoint, [[1/det_A]*n for _ in range(n)])

def generate_k_clique_dnf(n, k):
    edges = [(i, j) for i in range(n) for j in range(i+1, n)]
    clique_edges = random.sample(edges, k*(k-1)//2)
    dnf = []
    for edge_set in itertools.combinations(clique_edges, k):
        term = [0] * (n*(n-1)//2)
        for i, j in edge_set:
            term[i*n - i + j - 1] = 1
        dnf.append(term)
    return dnf

def generate_random_dnf(n, m):
    edges = [(i, j) for i in range(n) for j in range(i+1, n)]
    dnf = []
    for _ in range(m):
        width = random.randint(2, n*(n-1)//2)

```

```

        term = [0] * (n*(n-1)//2)
        selected_edges = random.sample(edges, width)
        for i, j in selected_edges:
            term[i*n - i + j - 1] = 1
        dnf.append(term)
    return dnf

def greedy_certifier(dnf, x):
    live_terms = [i for i, term in enumerate(dnf) if any(x[j] == 0 for j in range(len(term)))]
    cert_length = 0
    while len(live_terms) > 0:
        max_live_vars = sum([sum(1 for j in range(len(term)) if term[j] == 1 and x[j] == 0) for i, term in enumerate(dnf) if i in live_terms])
        var_to_reveal = -1
        for j in range(n*(n-1)//2):
            if any(x[j] == 0 for i, term in enumerate(dnf) if i in live_terms and term[j] == 1):
                num_live_terms = sum(1 for i, term in enumerate(dnf) if i in live_terms and term[j] == 1)
                if var_to_reveal == -1 or num_live_terms > max_live_vars:
                    max_live_vars = num_live_terms
                    var_to_reveal = j
        x[var_to_reveal] = 1
        cert_length += 1
        live_terms = [i for i, term in enumerate(dnf) if any(x[j] == 0 for j in range(len(term)))]
    return cert_length

def calculate_mu(F, n):
    D_k = [random.getrandbits(n*(n-1)//2) for _ in range(30)]
    mu = sum(greedy_certifier(F, x) for x in D_k) / len(D_k)
    return mu

n_values = [6, 7, 8, 9, 10]
k_values = [3, 4]
results = []

for n in n_values:
    m_values = [10, 30, n**2]

    F_clique = generate_k_clique_dnf(n, k)
    mu_clique = calculate_mu(F_clique, n)
    results.append({"metric_name": "mu_clique", "metric_value": mu_clique, "instances_tested": 1})

    for m in m_values:
        F_rand = generate_random_dnf(n, m)
        mu_rand = calculate_mu(F_rand, n)
        results.append({"metric_name": f"mu_rand_{m}", "metric_value": mu_rand, "instances_tested": 1})

    return {
        "TRIAL": results
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]
    all_results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

all_results.extend(result["TRIAL"])

mu_clique_values = [r["metric_value"] for r in all_results if r["metric_name"] == "mu_clique"]
mu_rand_values = [r["metric_value"] for r in all_results if r["metric_name"].startswith("mu_rand")]

mean_mu_clique = sum(mu_clique_values) / len(mu_clique_values)
std_mu_clique = math.sqrt(sum((x - mean_mu_clique) ** 2 for x in mu_clique_values) / len(mu_clique_values))
support_fraction_clique = sum(1 for r in all_results if r["metric_name"] == "mu_clique" and r["support"] > 0.95)

mean_mu_rand = sum(mu_rand_values) / len(mu_rand_values)
std_mu_rand = math.sqrt(sum((x - mean_mu_rand) ** 2 for x in mu_rand_values) / len(mu_rand_values))
support_fraction_rand = sum(1 for r in all_results if r["metric_name"].startswith("mu_rand") and r["support"] > 0.95)

if support_fraction_clique >= 0.95 and support_fraction_rand >= 0.95:
    RESULT = f"SUPPORTED mean_mu_clique={mean_mu_clique} std_mu_clique={std_mu_clique} support_fraction_clique={support_fraction_clique} support_fraction_rand={support_fraction_rand}"
elif any(not r["conjecture_holds"] for r in all_results):
    counterexample = next(r for r in all_results if not r["conjecture_holds"])["metric_name"]
    RESULT = f"FALSIFIED counterexample={counterexample} first_failing_seed={seeds[next(i for i, s in enumerate(seeds) if s == counterexample)]}"
else:
    RESULT = "INCONCLUSIVE reason=insufficient_support"

print(RESULT)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5c70bf99.py", line 136, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5c70bf99.py", line 118, in run_trial
    F_clique = generate_k_clique_dnf(n, k)
                                   ^
NameError: name 'k' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError ('k' is not defined') before producing any data, so neither the support nor falsification condition could be evaluated. | next: Fix the scoping bug in run_trial so k is passed into generate_k_clique_dnf, then rerun the full 30-seed sweep across $n \in \{6..10\}$, $k \in \{3,4\}$.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	181693
2	preregistration	claude_max	opus	0	0	7774
3	novelty	claude_max	opus	0	0	4103
4	novelty	claude_max	opus	0	0	13541
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24182
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	87378
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23279
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22503
9	judge	claude_max	opus	0	0	4816

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 369269 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/17e6b7435e5a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/17e6b7435e5a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/17e6b7435e5a.tar.gz` (if generated)